LAB 3 결과보고서

FPGA 응용회로 구현과 PC–FPGA UART 에코 보드 실험

과목	전자전기컴퓨터설계실험 2
이름 · 학번	김태이 · 2023440158
실험일 · 작성일	2026-09-28 · 2026-10-11
대상 소자	AMD Spartan-7 xc7s75fgga484-1 (HBE-Combo II-DLD)
검증 도구	Vivado 2025.2 시뮬레이터·구현, VS Code + Icarus Verilog 12
보드 시연	LAB3-07 PC–FPGA UART 에코 (조교 확인), 01–06은 영상으로 기록
GitHub · 커밋	github.com/tae2yi/UOS_ece2 · 6d7564f
시현 영상	github.com/tae2yi/UOS_ece2 · lab3/report/final/video

요약 PWM LED, RGB PWM, 피에조, 스텝모터, MM:SS 시계, 문자 LCD, UART 에코의 7개 FPGA 응용회로를 50 MHz 단일 클록과 카운터 기반 clock-enable 구조로 작성하였다. 같은 RTL과 자기검사 테스트벤치를 Vivado 시뮬레이터와 Icarus Verilog에서 실행하여 7개 모두 교안의 PASS 문자열과 검사 수를 얻었고, UART를 제외한 6개는 종료 시각까지 일치하였다. UART의 60 ns 차이는 테스트벤치 자극과 DUT 샘플링이 같은 클록 에지에서 일어나는 경쟁 조건 때문이며 검사 결과에는 영향이 없다. 7개 설계는 모두 DRC 오류 없이 구현되었고 최소 setup 여유(WNS)는 8.623 ns였다. 보드에서는 LAB3-07을 조교에게 시연하였다. Tera Term에서 입력한 문자가 한 번씩 되돌아왔고, LED가 A(0x41)부터 H(0x48)까지 입력 문자의 ASCII 코드로 바뀌었다. 나머지 6개도 보드에서 동작을 확인해 영상으로 남겼으며, 피에조 음은 영상 음성 분석에서 기본 주파수 약 294.0 Hz로 계산값과 일치하였다.

1 서론

1.1 실험 목적

LAB3의 목적은 PWM, 구동기, 표시장치, 직렬통신처럼 실제 부품을 다루는 FPGA 응용회로를 하나의 50 MHz 클록 위에서 설계하고, 같은 소스를 시뮬레이션, 구현, 보드의 세 단계로 검증하는 것이다 [1]. 느린 동작은 분주한 클록이 아니라 카운터가 만드는 clock-enable로 처리하고, 버튼·UART RX 같은 비동기 입력은 두 플립플롭으로 동기화한다. 보드 실험은 LAB3-07 PC–FPGA UART 에코로 수행하였다.

1.2 배경 이론

단일 클록과 clock-enable. 카운터의 끝 값에서 한 클록 폭의 enable을 만들고, 모든 레지스터는 `clk_50mhz`의 상승 에지에서만 갱신한다. 분주한 신호를 클록으로 쓰면 클록 경로가 늘어 타이밍 분석이 복잡해지고 클록 간 데이터 전달 문제가 생기므로 피한다 [1].

비동기 입력 동기화. 클록과 무관하게 바뀌는 입력은 첫 플립플롭에서 준안정 상태가 될 수 있다. 두 번째 플립플롭이 한 클록 동안 안정될 시간을 주어 이후 로직에는 0 또는 1만 전달되게 한다. 시뮬레이션은 준안정 자체가 아니라 동기화 이후의 디지털 순서를 검사한다 [1, 2].

UART 8N1. 유휴 상태는 1이고, 한 프레임은 start(0), 데이터 8비트(LSB 먼저), stop(1)의 10비트다. 수신기는 start 하강 에지를 기준으로 반 비트 뒤 start를 다시 확인하고, 이후 한 비트 간격마다 비트 중앙을 샘플한다 [3]. 송수신 baud가 다르면 샘플 위치가 비트마다 밀리는데, stop 비트 중앙(start 에지 후 9.5비트)까지 밀림이 반 비트보다 작아야 하므로 양쪽 합산 허용 오차는 0.5/9.5, 약 5 % 이하다.

1.3 가설과 예상 결과

- 7개 설계는 Vivado 시뮬레이터와 Icarus에서 같은 PASS 문자열과 검사 수를 낸다. 테스트벤치가 같은 RTL을 같은 자극으로 구동하기 때문이다.
- 모든 설계는 20 ns 클록 제약을 큰 여유로 만족한다. 카운터와 비교기 위주라 조합 경로가 짧다.
- UART 보드 실험에서 문자 하나를 입력하면 같은 문자가 한 번 되돌아오고, LED[7:0]에 그 ASCII 코드가 표시된다. FPGA의 baud 오차는 +0.0064 %로 허용 오차보다 훨씬 작아 framing error가 없을 것이다.
- 쉬지 않고 연속으로 보내면 송신 완료와 다음 수신 사이의 여유가 수 클록뿐이어서 일부 문자가 빠질 수 있다.

표 1. 실험 후 보고서 요구사항과 대응 위치.

실험 후 보고서 요구사항 [5]	본 보고서 대응
모든 실험의 Vivado 시뮬레이션 파형과 해석	3장 표 4, 그림 2–9
시연 실험의 보드 사진(실험 번호·입력 조건·확인한 동작)	6.3절 그림 11–13, 표 8
예상과 실제 결과 비교, 수정·재검증	4장 표 5, 6.4절, 7장
핵심 코드 제시	2.4절
GitHub 주소, 제출 태그와 커밋 해시	표지, 9장
허용된 출처 인용과 참고문헌 대응	본문 [1]–[5], 9장

2 In-Lab 실험 내용

2.1 실험 도구와 재료

표 2. 실험 도구와 재료.

구분	사용한 것
FPGA 보드	HBE-Combo II-DLD, Spartan-7 xc7s75fgga484-1, MAIN CLOCK F 50 MHz (B6)
PC 연결	USB 케이블, 직렬 터미널 Tera Term (COM5, 9600 8N1)
설계·검증	Vivado 2025.2 시뮬레이터, fpga-lab-template v2.0.2, VS Code, Icarus Verilog 12
구현·기록	Vivado 2025.2 합성·구현, Hardware Manager
소스 관리	Git, GitHub tae2yi/UOS_ece2

2.2 실험 절차

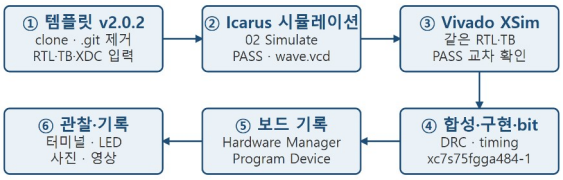


그림 1. 실험 절차. 7개 실험 모두 ①–⑥을 수행하였고, LAB3-07을 조교에게 시연하였다.

1) **프로젝트 생성.** 실험마다 템플릿 v2.0.2를 새 폴더로 받아 교안의 RTL·TB·XDC·simulation.json을 입력하였다.
2) **Icarus.** `02 Simulate`로 컴파일·실행하고 PASS 문자열, 검사 수, 종료 시각과 wave.vcd를 보관하였다.
3) **Vivado 시뮬레이션.** 같은 RTL·TB를 Vivado 시뮬레이터로 실행해 PASS를 확인하고 파형 창을 캡처하였다.
4) **구현.** Part xc7s75fgga484-1, 설계 top lab3_*로 합성·구현·bitstream을 만들고 DRC·timing·핀을 확인하였다.
5) **보드 기록.** Hardware Manager에서 Open Target → Auto Connect → Program Device로 실험별 bitstream을 기록하였다.
6) **관찰.** 실험별 입력을 주며 동작을 영상으로 촬영하였다. LAB3-07은 Tera Term에서 문자를 입력하며 에코와 LED를 확인하였다.

2.3 회로별 설계 요약

표 3. 7개 실험의 설계 top과 테스트벤치 검사 수.

실험	설계 top	핵심 동작	보드 파라미터	검사
01	lab3_led_pwm	버튼마다 duty 10 %↑, 8 LED	PWM 1 kHz, 20 ms 디바운스	4
02	lab3_rgb_pwm	R·G·B 독립 PWM	채널당 0..100 %	2
03	lab3_piezo	294 Hz 사각파	반주기 85,034 클록	5
04	lab3_stepper	4상 정·역회전·정지	10 ms/step	8
05	lab3_mmss_clock	BCD 시계, 4자리 스캔	1 s, 스캔 4 kHz	7
06	lab3_character_lcd	HD44780 초기화·2행 표시	tick 10 µs	40
07	lab3_uart_echo	9600 8N1 에코 + LED	DIV 5,208클록	3

각 회로의 블록 흐름, 파라미터 계산, RTL·TB·XDC 역할과 TB 자극은 예비보고서 [4]에 정리하였으므로 여기서는 결과 위주로 기술한다.

2.4 핵심 코드

7개 회로는 모두 「카운터가 끝 값에 닿으면 한 클록 동안 동작한다」는 같은 뼈대를 쓴다. 시연한 LAB3-07에서 그 뼈대가 가장 잘 드러나는 두 부분을 싣는다. 수신기는 `timer`가 0이 될 때만 상태를 옮긴다. 상태 0에서 start 하강을 보면 반 비트(DIV/2)를 기다려 start를 다시 확인하고, 이후 DIV마다 비트 중앙에서 `sync[1]`을 `shift[bitno]`에 담는다. stop이 1이면 `valid`를 한 클록 올린다.

```
0: if(!sync[1]) begin timer<=DIV/2-1; state<=1; end
1: if(timer!=0) timer<=timer-1;
   else if(sync[1]) state<=0;
   else begin timer<=DIV-1; bitno<=0; state<=2; end
2: if(timer!=0) timer<=timer-1;
   else begin
     shift[bitno]<=sync[1]; timer<=DIV-1;
     if(bitno==7) state<=3; else bitno<=bitno+1'b1;
   end
3: if(timer!=0) timer<=timer-1;
   else begin
     if(sync[1]) begin data<=shift; valid<=1; end
     else framing_error<=1;
     state<=0;
   end
```

GitHub
https://github.com/tae2yi/UOS_ece2/blob/6d7564f/lab3/lab3_07_uart_echo/src/uart.v#L20-L35

top은 `rx_valid`가 뜬 클록에 수신 값을 `last_data`(LED)에 저장하고, 송신기가 비어 있으면(`tx_ready`) 같은 값을 `tx_valid` 한 클록과 함께 넘겨 에코한다.

```
tx_valid <= 0;
if (rx_valid) begin
  last_data <= rx_data;
  if (tx_ready) begin
    tx_data <= rx_data;
    tx_valid <= 1;
  end
end
...
assign led = last_data;
```

GitHub
https://github.com/tae2yi/UOS_ece2/blob/6d7564f/lab3/lab3_07_uart_echo/src/lab3_uart_echo.v#L32-L42

3 기능 시뮬레이션 결과

표 4는 두 시뮬레이터의 결과다. 두 도구 모두 TB 클록 20 ns(보드와 같음)에 시간 상수만 줄인 파라미터를 사용하였다. 아래 그림 2–9는 Vivado 2025.2 시뮬레이

터로 TB를 실행한 뒤 Vivado의 파형 창을 캡처한 것이다. 노란 세로선은 시뮬레이션이 끝난 시각이고, 값은 표시가 없으면 16진수다.

표 4. Vivado 시뮬레이터와 Icarus의 PASS 문자열과 종료 시각.

실험	PASS 문자열	Icarus (ns)	Vivado (ns)	비교
01	LAB3_LED_PWM_PASS checks=4	3831	3831	일치
02	LAB3_RGB_PWM_PASS checks=2	4431	4431	일치
03	LAB3_PIEZO_PASS edges=5	551	551	일치
04	LAB3_STEPPER_PASS checks=8	831	831	일치
05	LAB3_MMSS_PASS checks=7	144031	144031	일치
06	LAB3_LCD_PASS bytes=40	8130	8130	일치
07	LAB3_UART_ECHO_PASS checks=3	9970	9910	PASS 일치, 60 ns 차

3.1 LAB3-01 PWM LED

버튼 11회 입력에 따라 level이 0→3→10→0으로 바뀐다. pwm의 HIGH 폭은 level과 같은 클록 수로 늘어나고, level 10에서는 주기 내내 HIGH, 다음 누름에서 다시 0이 된다. led[7:0]은 pwm을 8개에 복사한 것이라 00과 ff만 오간다.

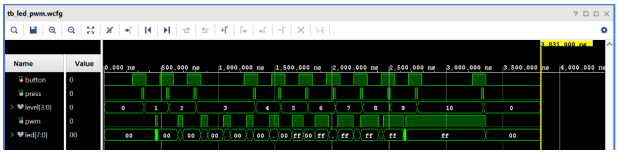


그림 2. LAB3-01 Vivado 파형(전체 3,831 ns). button 입력마다 level이 1씩 올라 10에 이르고, 다음 누름에서 0이 된다.

3.2 LAB3-02 RGB LED PWM

버튼을 R 2회, G 5회, B 8회 누르면 level_r·g·b가 2·5·8이 되고, 측정 구간 10클록에서 pwm_r·pwm_g·pwm_b의 HIGH가 각각 2·5·8클록이다. 이어 R을 한 번 더 눌러 3·5·8을 확인한다. 파형 오른쪽으로 갈수록 pwm_b의 폭이 넓어지는 것이 보인다.

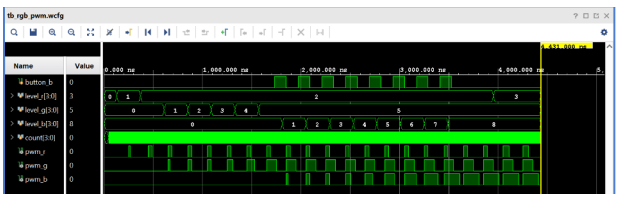


그림 3. LAB3-02 Vivado 파형(전체 4,431 ns). 채널별 level과 PWM 폭이 독립으로 바뀐다.

3.3 LAB3-03 피에조

count가 0..4를 돌고 끝 값마다 piezo가 반전되어 100 ns(5클록)마다 바뀌는 50 % 사각파가 된다. 보드 파라미터에서는 같은 구조로 반주기 85,034클록, 294.000 Hz가 된다.

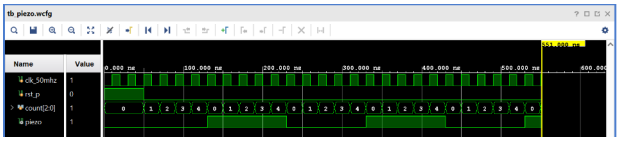


그림 4. LAB3-03 Vivado 파형. count 0–4, 반주기 5클록, 551 ns에 종료.

3.4 LAB3-04 스텝모터

enable이 켜지면 4클록마다 state가 0→1→2→3→0으로, direction=1에서는 3→2로 이동한다. enable=0 구간에서는 1100이 유지되어 정지 시 마지막 코일 패턴을 붙잡는다.

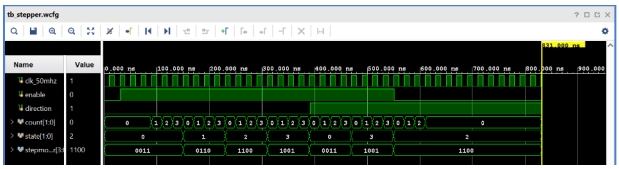


그림 5. LAB3-04 Vivado 파형. stepmotor(2진)가 정방향 한 주기, 역방향 2단계 뒤 1100에서 멈춘다.

3.5 LAB3-05 MM:SS 시계

subsecond가 0..1을 도는 동안 초 1의 자리가 2클록마다 오르고, 9 다음에 0으로 돌아가며 초 10의 자리가 1이 된다(431 ns, 00:10). 테스트벤치는 이후 01:00, 59:59, 00:00까지 3,600초를 모두 확인하였다.

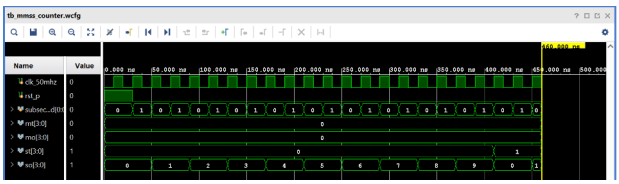


그림 6. LAB3-05 Vivado 파형 앞부분(0–460 ns). so가 9 다음 0이 되며 st가 1이 된다.

3.6 LAB3-06 문자 LCD

lcd_e가 내려가는 순간 TB가 RS와 데이터를 비교한다. lcd_data를 ASCII로 표시하면 초기화 명령 뒤에 1행 「FPGA LAB3」, 줄바꿈 명령, 2행 「LCD CONTROLLER」가 차례로 지나가는 것이 그대로 읽힌다. lcd_rs는 문자

구간에서만 1이다. 40번째 바이트 확인 후 8,130 ns에 PASS가 출력된다.

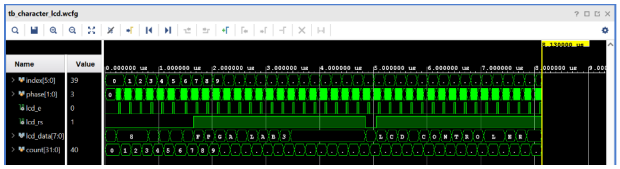


그림 7. LAB3-06 Vivado 파형(전체 8,130 ns). lcd_data(ASCII)에 두 줄의 문자열이 보인다.

3.7 LAB3-07 UART 에코

그림 8은 첫 바이트 0x41(0100_0001)의 수신 과정이다. start(70–230 ns) 뒤 D0=1, D1–D5=0, D6=1, D7=0이 160 ns(DIV=8)씩 이어지고, bitno가 0→7로 오른다. stop 비트 중앙에서 rx_valid가 한 클록 올라가며 rx_data=41이 된다. 그림 9의 전체 구간에서는 세 바이트 0x41·0x5A·0x0A가 차례로 수신·에코되고 LED가 각 값으로 바뀐다.

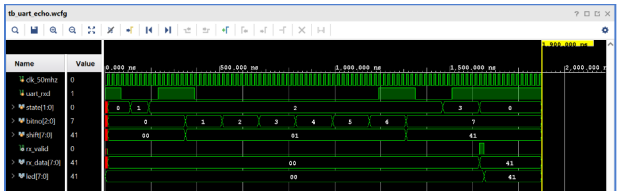


그림 8. LAB3-07 Vivado 파형, 첫 프레임 0x41(0–1,900 ns). 수신기 state(0 대기, 1 start 확인, 2 데이터, 3 stop)와 bitno, shift.

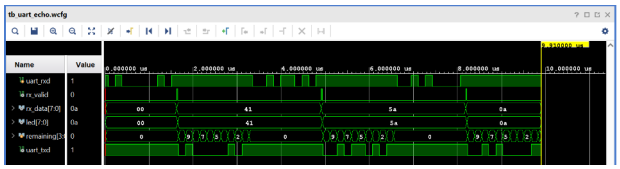


그림 9. LAB3-07 Vivado 전체 파형(9,910 ns). 수신(uart_rxd) 뒤에 에코(uart_txd)가 이어진다.

3.8 UART 종료 시각 60 ns 차이의 원인

TB는 `@(posedge clk)` 직후 blocking 대입으로 uart_rxd를 바꾼다. 같은 상승 에지에서 DUT의 동기화 플립플롭이 바뀌기 전 값을 읽을지 바뀐 뒤 값을 읽을지는 시뮬레이터의 이벤트 처리 순서에 달려 있다. Vivado 시뮬레이터는 새 값을 읽어 110 ns에, Icarus는 이전 값을 읽어 130 ns에 start를 인식하였다. 바이트당 1클록 차이가 세 바이트에서 60 ns로 쌓였다. receive_byte가 uart_txd 하강 에지에 다시 맞추므로 두 도구 모두 PASS다.

4 수정실험 결과

표 5. 수정실험 결과. 두 시뮬레이터에서 같은 결과를 얻었다.

실험	수정 (한 항목)	결과	복구
01	LEVELS 10→5	FAIL level=3 high=6	PASS
02	R 초기 duty 0→30 %	FAIL rgb high=5,5,8	PASS
03	TONE_HZ 100→125	FAIL half period=4	PASS
04	STEP_HZ 2→1	PASS, 831→1311 ns	PASS
05	TB CLK_HZ 2→4	FAIL time=00:05	PASS
06	2행 R→S	FAIL index=37 data=53	PASS
07	BAUD 100→200 (양쪽)	PASS, 9970→5170 ns	PASS

TB 기대값을 바꾸지 않았으므로 FAIL은 변경된 동작과 기대가 처음 갈라지는 시점에서 났다. 04와 07은 순서와 데이터가 유지되는 수정이라 PASS가 정상이고, 간격 변화는 종료 시각과 파형으로 확인하였다. 예상값·실제·복구의 3행 표와 비교 파형은 예비보고서 [4]에 있다.

5 합성·구현과 bitstream

7개 설계를 Part xc7s75fgga484-1로 합성·구현하고 bitstream을 생성하였다. 모든 XDC는 top 포트 전부에 PACKAGE_PIN과 LVCMOS33을 지정하고, Report Clocks에서 clk_50mhz 주기 20.000 ns를 확인하였다.

표 6. 구현 결과. 모든 설계에서 DRC 오류 0, bitstream 생성 완료.

실험	LUT	FF	IOB	WNS (ns)	WHS (ns)
01	92	52	11	9.197	0.131
02	276	144	17	8.623	0.080
03	21	18	3	16.761	0.246
04	20	25	8	16.038	0.160
05	83	58	18	15.379	0.271
06	111	93	13	14.243	0.219
07	134	118	12	14.921	0.122

가장 큰 02도 LUT 276개(0.58 %)로 소자의 1 %에 못 미친다. setup 여유가 가장 작은 02도 8.6 ns로, 20 ns 주기의 43 %가 남는다. hold 여유(WHS)는 모두 양수다. DRC 보고서의 경고는 모든 설계에서 CFGBVS-1 하나로, XDC에 설정 뱅크 전압 (CFGBVS,

CONFIG_VOLTAGE)을 지정하지 않았다는 뜻이며 동작에는 영향이 없다.

합성 경고는 05와 06에만 있었다. 05는 8자리 중 4자리만 쓰므로 seg_com[7:4]가 1로 고정되고, 06은 write-only라 lcd_rw가 0으로 고정되며 FSM의 쓰이지 않는 상태 비트 하나가 제거되었다. 모두 설계 의도대로다.

6 보드 실험

7개 bitstream을 모두 보드에 기록해 동작을 확인하고 영상으로 남겼다 [6]. 실습 시간에 조교에게 시연한 실험은 LAB3-07 UART 에코이므로 6.1–6.4절에서 자세히 다루고, 나머지 6개는 6.5절에 정리한다. 아래 사진은 모두 그 영상에서 캡처한 장면이다.

6.1 LAB3-07 실험 구성

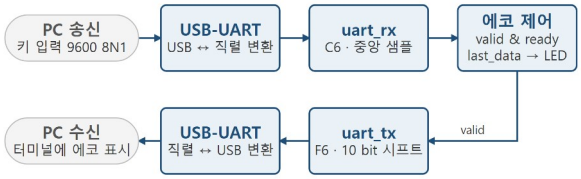


그림 10. UART 에코 실험 구성. PC 송신 → FPGA 수신 → 에코 송신 → PC 수신, 수신값은 LED로도 표시.

PC의 Tera Term에서 보낸 문자는 USB-UART를 거쳐 FPGA의 C6(uart_rxd)으로 들어오고, FPGA가 F6(uart_txd)로 같은 바이트를 돌려보낸다. 보드의 LED 실크는 LED1이 led[7], LED8이 led[0]이므로 왼쪽이 상위 비트다.

표 7. LAB3-07 핀 배치 (I/O 보고서와 일치).

포트	핀	방향	역할
clk_50mhz	B6	입력	50 MHz, 20 ns create_clock
rst_p	K4	입력	리셋 (false path)
uart_rxd	C6	입력	PC → FPGA (false path)
uart_txd	F6	출력	FPGA → PC
led[0]..[7]	N5·M1·M3·M7·N7·M2·M4·L4	출력	마지막 수신값 (LED8..LED1)

6.2 측정 절차

1) USB 케이블과 보드 전원을 연결하고 Program Device로 lab3_uart_echo.bit를 기록하였다.
2) Tera Term에서 보드의 직렬 포트(COM5)를 9600 baud, 데이터 8, 패리티 없음, stop 1로 열었다.
3) 문자를 하나씩 입력하며 터미널에 표시되는 문자와 LED1–8을 확인하고 영상으로 촬영하였다.

6.3 관찰 결과

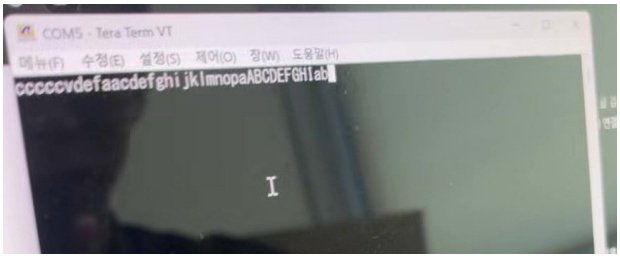


그림 11. LAB3-07 터미널 화면. 입력 조건: Tera Term COM5에서 영문 소문자·대문자를 차례로 입력. 확인한 동작: 입력한 문자가 한 번씩 표시된다.

그림 11에서 Tera Term 창에는 입력한 순서대로 `...defghijklmnopaABCDEFGHIab`가 한 글자씩 표시되어 있다. PC가 보낸 문자를 FPGA가 한 번 되돌려 보낸 결과와 일치하며, 같은 문자가 두 번씩 찍히는 현상은 없었다. 촬영 중 마지막에 입력한 a도 곧바로 화면에 나타났다.

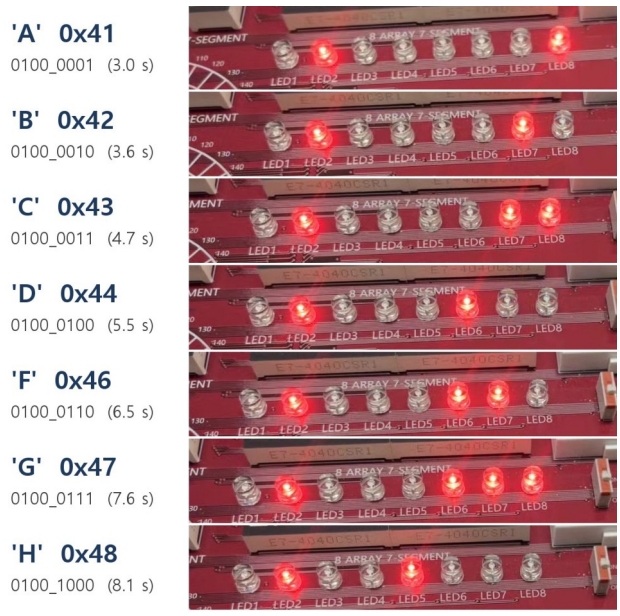


그림 12. LAB3-07 LED 변화. 입력 조건: Tera Term에서 대문자를 차례로 입력. 확인한 동작: LED1–8(LED1 = bit 7)이 각 문자의 ASCII 코드를 표시한다. 괄호 안은 영상의 시각이다.

표 8. LAB3-07 문자별 예상 LED와 영상에서 읽은 LED.

입력	ASCII	예상 LED1..8	켜진 LED (관찰)	판정
'A'	0x41	0100_0001	LED2, LED8	일치
'B'	0x42	0100_0010	LED2, LED7	일치
'C'	0x43	0100_0011	LED2, LED7, LED8	일치
'D'	0x44	0100_0100	LED2, LED6	일치
'F'	0x46	0100_0110	LED2, LED6, LED7	일치
'G'	0x47	0100_0111	LED2, LED6, LED7, LED8	일치
'H'	0x48	0100_1000	LED2, LED5	일치

LED는 영상의 3.0초부터 8.2초까지 0x41에서 0x48까지 순서대로 바뀌었고, 일곱 값 모두 예상 비트열과 일치하였다. 대문자의 공통 상위 비트 0100 때문에 LED2는 계속 켜져 있고 하위 4비트(LED5–8)만 2진수로 올라간다. D 다음에는 E(0100_0101) 없이 바로 F가 나타났다. 영상의 이 구간에는 터미널이 찍히지 않아 E를 입력하지 않은 것인지는 화면만으로 단정할 수 없으나, 30 fps의 모든 프레임에서 LED6·8 조합은 보이지 않았다.

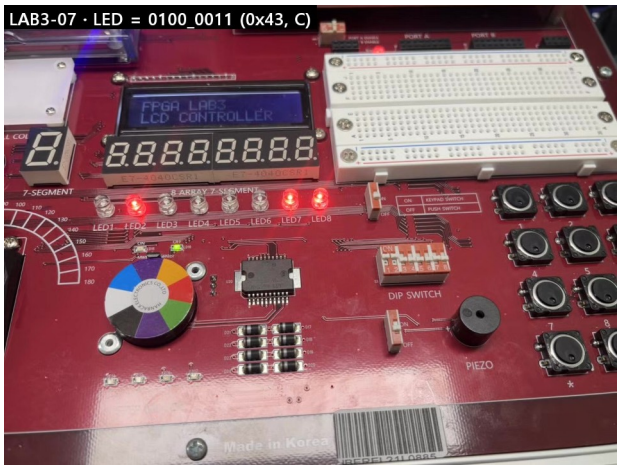


그림 13. LAB3-07 보드 전체. 입력 조건: 'C' 입력 직후. 확인한 동작: LED2·7·8이 켜져 0100_0011(0x43)을 표시한다. LCD의 글자는 앞서 기록한 LAB3-06의 표시가 남아 있는 것으로 보인다.

6.4 계산값과 관찰의 비교

표 9. UART 타이밍의 계산값과 보드 관찰.

항목	계산값	보드 관찰
DIV	(50,000,000+4,800)/9,600 = 5,208	—
FPGA baud	50 MHz / 5,208 = 9,600.6	9600 설정의 PC와 통신 성립
PC와의 오차	+0.0064 % (허용 약 5 %)	확인한 문자 모두 올바른 코드
1 bit / 1 frame	104.16 µs / 1.0416 ms	직접 측정하지 않음
LED 표시	마지막 수신 바이트	A–H 7개 값 일치

비트 시간은 오실로스코프로 재지 않았으므로 baud는 간접적으로만 확인하였다. baud가 크게 어긋나면 비트가 밀려 다른 코드가 수신되는데, 확인한 일곱 문자가 모두 올바른 코드로 LED에 표시되었고 터미널에도 입력한 문자가 그대로 돌아왔으므로 송수신 양쪽의 비트 시간이 허용 범위 안에서 맞았다고 볼 수 있다.

6.5 LAB3-01–06 보드 동작

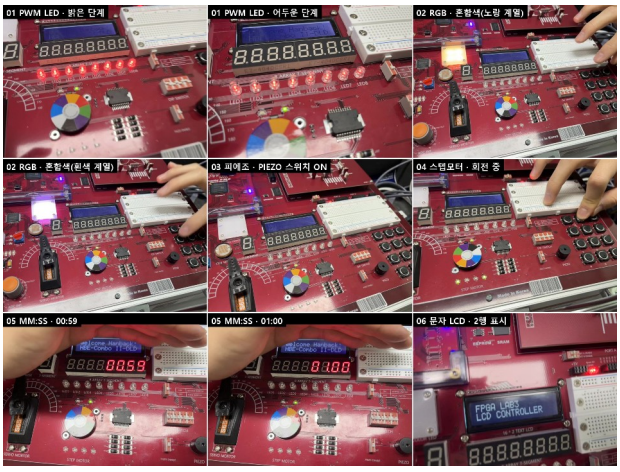


그림 14. LAB3-01–06 보드 동작(영상 캡처). 각 장면의 왼쪽 위에 실험 번호와 상태를 적었다.

표 10. LAB3-01–06의 입력 조건과 영상에서 확인한 동작.

실험	입력 조건	확인한 동작
01	버튼 반복 누름	LED1–8이 함께 단계적으로 밝아지고 최대 다음에 꺼짐
02	R·G·B 버튼	Full Color LED가 빨강, 노랑 계열, 흰색 계열로 바뀜
03	PIEZO 스위치 ON	단일 음 발생, 음성 분석 기본 주파수 약 294.0 Hz
04	구동·방향 버튼	색 원판 회전, 상 표시 LED 점등
05	없음	00:54부터 1초마다 증가, 00:59 → 01:00 자리올림
06	없음	1행 FPGA LAB3, 2행 LCD CONTROLLER

03은 소리라서 사진으로 남지 않으므로 영상의 음성을 주파수 분석하였다. 스위치를 올린 뒤 2초 구간의 스펙트럼 피크가 2,057.9, 2,351.9, 2,645.9, …, 4,703.9 Hz로 294 Hz의 7–16배에 정확히 놓였고, 가장 큰 피크 2,645.88 Hz를 9로 나누면 293.99 Hz다. 계산값 50 MHz/(2×85,034) = 294.000 Hz와 일치한다. 기본파보다 고조파가 크게 녹음된 것은 사각파에 고조파가 많고 피에조와 휴대전화 마이크가 수 kHz 대역에서 더 민감하기 때문으로 보인다. 05는 영상의 1초 간격 장면에서 초 자리가 하나씩 올랐고 59 다음에 분으로 자리 올림하였다.

7 토론

7.1 예상 결과와의 비교

시뮬레이션과 구현은 1.3절의 가설과 일치하였다. 7개 설계 모두 두 시뮬레이터에서 같은 PASS와 검사 수를 냈고, 모든 구현이 20 ns 제약을 8.6 ns 이상의 여유로 만족하였다. 보드에서도 가설 3대로 입력한 문자가 한 번씩 되돌아오고 LED에 그 ASCII 코드가 표시되었다. 가설 4(쉬지 않는 연속 입력에서의 누락)는 사람이 타이핑하는 속도로만 시험하였으므로 이번 실험으로는 확인하지 못하였다. 타이핑 간격은 한 프레임 1.04 ms보다 훨씬 길어 송신이 끝난 뒤에 다음 문자가 들어오기 때문이다.

7.2 차이가 생긴 원인

시뮬레이터 간 종료 시각. UART만 60 ns 달랐고 원인은 3.8절의 경쟁 조건이다. RTL의 결함이 아니라 TB가 DUT와 같은 에지에서 입력을 바꾸는 방식 때문이다.

baud 양자화. 50 MHz를 9600으로 나눈 5,208.33을 5,208로 반올림하여 +0.0064 %의 오차가 생긴다. 한 프레임 동안 샘플 위치는 약 3클록(60 ns)만 밀려 비트 폭 104 µs에 비해 무시할 수 있다.

보드에서 생길 수 있는 차이. 터미널의 local echo가 켜져 있으면 같은 문자가 두 번 보이고, Enter는 설정에 따라 0x0D 또는 0x0D·0x0A로 보내져 LED에 마지막 값만 남는다. 이번 시연에서는 두 현상 모두 나타나지 않았다. 다만 LED는 마지막 한 바이트만 보여 주므로, 빠르게 입력하면 중간 값은 눈으로 따라가기 어렵다.

7.3 개선 방법

- TB 자극을 클록 하강 에지에서 바꾸거나 `#1` 지연을 두면 경쟁 조건이 사라져 두 시뮬레이터의 시각이 일치한다.
- 에코 경로에 1바이트 버퍼를 두면 송신 중에 들어온 바이트도 잃지 않아 연속 입력에서도 누락이 없다.
- XDC에 CFGBVS와 CONFIG_VOLTAGE를 지정하면 DRC 경고가 사라진다.

7.4 실험 방법의 문제점과 제안

04와 07의 TB는 순서와 데이터만 검사하므로 step 간격이나 baud가 바뀌어도 PASS가 난다. 간격을 클록 수로 측정해 비교하는 검사를 추가하면 수정실험에서도 변화를 자동으로 잡을 수 있다. 05의 TB는 카운터와 디코더만 검사하고 top의 스캔 MUX는 검사하지 않으므로 seg_com 순서를 확인하는 검사가 필요하다. 또한 교안은 Vivado 2026.1 기준이지만 본 실험은 2025.2로 수행하였고, 7개 설계에서 버전 차이로 인한 문제는 없었다. 보드 실험에서는 비트 시간을 오실로스코프로 재지 않아 baud를 간접적으로만 확인한 점이 아쉽다. uart_txd를 프로브로 측정하면 104.16 µs와 직접 비교할 수 있다.

8 결론

7개 FPGA 응용회로를 50 MHz 단일 클록과 clock-enable 구조로 작성하고, Vivado 시뮬레이터와 Icarus에서 같은 PASS를 확인한 뒤 모두 DRC 오류 없이 20 ns 타이밍을 만족하도록 구현하였다. 같은 카운터 구조라도 끝 값이 주기, 반주기, step 간격, 비트 시간 중 무엇을 뜻하는지에 따라 회로의 성격이 달라지며, 그 값을 파라미터로 두면 시뮬레이션에서는 작게, 보드에서는 실제 값으로 같은 RTL을 쓸 수 있다는 점을 확인하였다. 시뮬레이터마다 종료 시각이 달라진 UART 사례는 TB 자극의 시점도 설계만큼 중요하다는 것을 보여 주었다. 보드에서는 UART 에코를 시연하여 Tera

Term에 입력한 문자가 그대로 돌아오고 LED가 A부터 H까지의 ASCII 코드를 차례로 표시하는 것을 확인하였다. 나머지 6개 회로도 보드에서 동작하였고, 피에조 음은 음성 분석으로 약 294.0 Hz임을 확인하였다.

9 제출 자료와 참고문헌

GitHub 저장소.

https://github.com/tae2yi/UOS_ece2/tree/master/lab3

제출 태그 · 소스 commit. lab3-final · 6d7564f1bb7daf042e26bdb3fd0ad70fbc7b45a9

시연 영상.

https://github.com/tae2yi/UOS_ece2/tree/master/lab3/report/final/video

자료 위치. 실험 원본은 lab3/lab3_0N_*/의 src·sim·constraints·simulation.json이다. 예비보고서와 수정실험 자료는 lab3/report/pre/, 이 보고서와 그림은 lab3/report/final/, 실험 영상 7개와 설명은 lab3/report/final/video/에 있다. 태그 lab3-final은 이 보고서와 영상을 포함한 제출 버전을 가리킨다.

[1] 이해리,「LAB 3 · FPGA 응용회로 공통 예습」및「LAB3-25 PC–FPGA UART 에코」, 서울시립대학교 전자전기컴퓨터설계실험II 교안, 2026.

[2] D. M. Harris, S. L. Harris, Digital Design and Computer Architecture, 2nd ed., Morgan Kaufmann, 2012, 3.5절(동기화기). ISBN 978-0-12-394424-5.

[3] P. P. Chu, FPGA Prototyping by Verilog Examples, Wiley, 2008, 8장(UART). ISBN 978-0-470-18532-2.

[4] 김태이, LAB3 예비보고서, 2026-09-28 (lab3/report/pre/LAB3_예비보고서.docx).

[5] 이해리,「LAB 0. 오리엔테이션」, 서울시립대학교 전자전기컴퓨터설계실험II, 2026-09-07, pp.32–37.

[6] 김태이, LAB 3 보드 실험 영상, GitHub tae2yi/UOS_ece2, lab3/report/final/video/README.md.